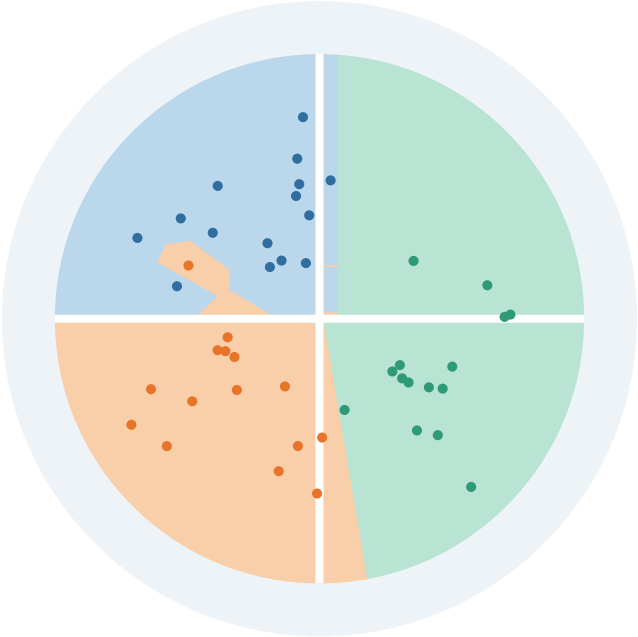
Модели изнутри

Урок



Модель — правило, которое по описанию объекта выдаёт ответ. Здесь четыре семейства моделей: ближайшие соседи, решающие деревья, линейные модели и k-средних. Каждое разобрано изнутри: что оно считает, как учится и где ломается, и всё написано на numpy. В последнем разделе то же делают готовые классы sklearn, и видно, где их правила расходятся с точными. Код — в цветных ячейках, под чертой — вывод, снятый запуском; ячейки идут подряд, как в ноутбуке.

1 Что такое модель

Объект — то, о чём спрашивают: пингвин, письмо, квартира. Он описан d числами, *признаками*, то есть объект — вектор $x \in \mathbb{R}^d$. У объекта есть *ответ* y: вид пингвина, спам или нет, цена. Если ответ — одна из нескольких меток, задача называется *классификацией*, а метки — *классами*; их нумеруют $0, 1, \ldots, K-1$. Если ответ — число, это *регрессия*.

Пример на весь урок — двенадцать пингвинов трёх видов: Адели (класс 0), антарктический (1) и папуанский (2). Числа придуманы, но похожи на настоящие. Признаки — длина клюва в миллиметрах и масса в граммах. Объекты — строки таблицы X размера $n \times d$, ответы — вектор y:

```
import numpy as np
# клюв, мм; масса, г
X = np.array([[38, 3600], [40, 3800], [36, 3300],
              [41, 3950], [47, 3550], [50, 3800],
              [46, 3400], [51, 3950], [46, 4900],
              [49, 5400], [41, 4650], [50, 5600]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2])
print(X.shape, np.bincount(y))
```

```
(12, 2) [4 4 4]
```

Модель — функция $a(x)$, которая объекту ставит в соответствие ответ. Её не пишут руками, а строят по примерам с известными ответами — это *обучение*, а таблица примеров — *обучающая выборка*, по-английски train. Потом модель отвечает на новых объектах — это *предсказание*. Кроме того, что модель выучила, у неё обычно есть *гиперпараметры* — то, что задают до обучения: число соседей, глубина дерева.

Хорошая модель верно отвечает на новых объектах, а не на тех, что видела. Поэтому качество меряют на *тестовой выборке*, test: её откладывают заранее и в обучении не используют. Шесть пингвинов, которых модели не увидят до проверки:

```
Xt = np.array([[41, 3520], [39, 3880], [48, 3700],
               [45, 3450], [47, 5150], [44, 4750]])
yt = np.array([0, 0, 1, 1, 2, 2])
```

Качество классификации — *доля верных ответов*, accuracy, а регрессии — *среднеквадратичная ошибка*, MSE:

$$\text{acc} = \frac{1}{n}\sum_{i=1}^{n}[a(x_i) = y_i], \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\big(a(x_i) - y_i\big)^2,$$

где $[\dots]$ — 1, если условие верно, и 0, если нет. Самая простая модель ничего не учит и всегда отвечает одно и то же. Это точка отсчёта, baseline: модель, которая его не обходит, бесполезна.

```
pred = np.zeros(6, dtype=int)
print((pred == yt).mean())
```

```
0.3333333333333333
```

Ловушка. Качество на обучающей выборке ничего не говорит о новых объектах. Модель «увидел знакомый объект — верни его ответ, иначе 0» на train безошибочна,

а на test не лучше константы. Чем гибче модель, тем легче ей подогнаться под train вместе с его случайностями. Это *переобучение*, и видно оно только на отложенных данных.

2 Ближайшие соседи

Похожие объекты обычно отвечают похоже. Модель k *ближайших соседей* так и устроена: для объекта q она находит k обучающих объектов, ближайших к q, и отдаёт ответ, которого среди них больше всего. Обучения как такового нет: модель — сама таблица.

Близость меряют расстоянием. Обычное, *евклидово*, или L2, и *манхэттенское*, L1, — путь, когда ходить можно только вдоль осей:

$$\rho_2(x,q) = \sqrt{\sum_{j=1}^{d}(x_j - q_j)^2}, \qquad \rho_1(x,q) = \sum_{j=1}^{d}|x_j - q_j|.$$

Расстояния от первого пингвина теста до всех двенадцати — через распространение, без цикла:

```
q = Xt[0]
d2 = np.sqrt(((X - q) ** 2).sum(axis=1))
d1 = np.abs(X - q).sum(axis=1)
print(d2.round(1))
print(d1)
```

```
[  80.1  280.   220.1  430.    30.6  280.1  120.1
  430.1 1380.  1880.  1130.  2080. ]
[  83  281  225  430   36  289  125  440 1385 1888
 1130 2089]
```

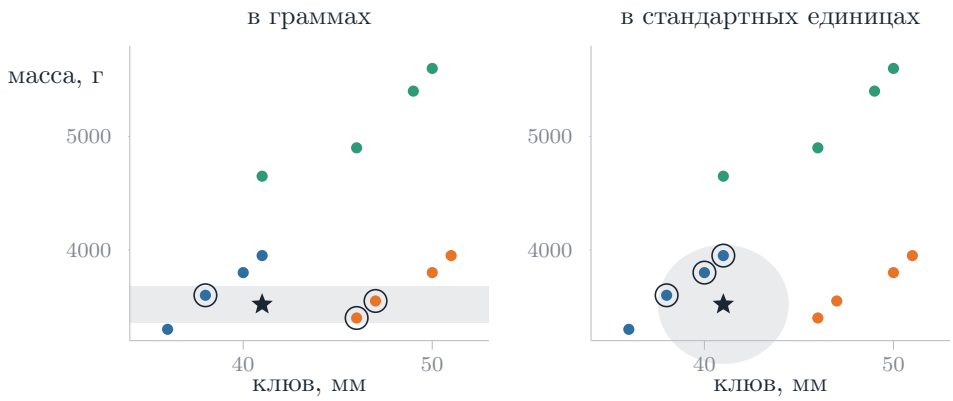
Соседи — первые k номеров по возрастанию расстояния, `argsort`. Голосование — счёт классов среди соседей, `bincount`, и номер самого частого, `argmax`:

```
k = 3
nb = np.argsort(d2, kind="stable")[:k]
votes = np.bincount(y[nb], minlength=3)
print(nb, y[nb], votes, votes.argmax())
```

```
[4 0 6] [1 0 1] [1 2 0] 1
```

Соседи — пингвины 4, 0 и 6, голоса 1 : 2 : 0, ответ — класс 1. Если голоса равны, `argmax` берёт меньший номер класса; так решает и sklearn, но это лишь одно из возможных правил, и в задаче правило ничьей задаёт условие.

Ответ — антарктический, а пингвин — Адели. Причина видна в выводе: оба расстояния — почти одна разность масс. Масса различается на сотни граммов, клюв — на единицы миллиметров, и клюв, по которому Адели и отличается от антарктического, в расстоянии почти не участвует. На рисунке слева — кого модель считает близким в граммах: всех, чья масса около 3520, при любом клюве.



Масштаб признаков

Стандартизация: из каждого признака вычесть среднее и поделить на стандартное отклонение,

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad \sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \mu_j)^2}.$$

У каждого признака после неё среднее 0 и разброс 1, и разница на единицу значит одно и то же по любой оси. *Min-max* переводит признак в отрезок $[0, 1]$: $z_j = (x_j - m_j)/(M_j - m_j)$, где m_j и M_j — наименьшее и наибольшее значения.

> **Правило.** Среднее и разброс — или наименьшее и наибольшее — считают по train и теми же числами преобразуют test. Тест — это будущие объекты, их статистики при обучении неизвестны; а модель, обученная в одних единицах, должна получать новые объекты в тех же.

```
mu, sd = X.mean(axis=0), X.std(axis=0)
Z = (X - mu) / sd
Zt = (Xt - mu) / sd
print(mu.round(1), sd.round(1))
print(Zt[0].round(2))
```

```
[  44.6 4158.3] [  4.9 750. ]
[-0.73 -0.85]
```

Теперь соседи для всех шести пингвинов теста, в граммах и в стандартных единицах. Цикл здесь по объектам теста, а внутри — те же три строки:

```
def knn(Xtr, ytr, q, k):
    d = np.sqrt(((Xtr - q) ** 2).sum(axis=1))
    nb = np.argsort(d, kind="stable")[:k]
    return np.bincount(ytr[nb]).argmax()

for A, B in [(X, Xt), (Z, Zt)]:
    pred = np.array([knn(A, y, q, 3) for q in B])
    print(pred, (pred == yt).mean())
```

```
[1 0 0 1 2 2] 0.6666666666666666
[0 0 1 1 2 2] 1.0
```

В граммах модель ошиблась на двух пингвинах из шести, в стандартных единицах — ни на одном. Справа на рисунке выше — те, кто близок к первому пингвину теста после стандартизации: эллипс, а не полоса.

Регрессия и веса

Для регрессии голосование заменяют средним: ответ — среднее y у k соседей. Масса пингвина по длине крыла в миллиметрах; три соседа для крыла 205 мм:

```
fl = np.array([185, 190, 182, 195, 193, 198,
               189, 201, 214, 221, 210, 224])
m = X[:, 1]
nb = np.argsort(np.abs(fl - 205), kind="stable")[:3]
print(fl[nb], m[nb], m[nb].mean().round(1))
```

```
[201 210 198] [3950 4650 3800] 4133.3
```

Все k соседей голосуют одинаково, хотя ближний похож на q сильнее дальнего. Во *взвешенных соседях* голос соседа x_i — вес w_i, убывающий с расстоянием, например $w_i = 1/\rho(x_i, q)$.

Класс — тот, у которого больше сумма весов, а в регрессии ответ — взвешенное среднее:

$$a(q) = \arg\max_{c} \sum_{i \in N_k(q)} w_i \,[y_i = c], \qquad a(q) = \frac{\sum_{i \in N_k(q)} w_i y_i}{\sum_{i \in N_k(q)} w_i},$$

где $N_k(q)$ — номера k ближайших к q. При равных весах это обычное большинство и обычное среднее.

Выбор k и скользящий контроль

k — гиперпараметр. При $k = 1$ модель повторяет каждую случайность train: один нетипичный пингвин отвоёвывает себе окрестность. При $k = n$ соседи — все, и модель — константа. Хорошее k где-то между, и выбирать его по test нельзя: test тогда тоже станет обучающим, и его оценка перестанет быть честной.

Выбирают *скользящим контролем*, по-английски cross-validation. Train делят на m частей; каждую по очереди откладывают, модель учат на остальных и меряют на отложенной; итог — среднее m замеров. Каждый объект побывает отложенным ровно раз. Крайний случай $m = n$ — *контроль по одному*, leave-one-out: каждый объект предсказывается по всем остальным,

$$\text{LOO} = \frac{1}{n} \sum_{i=1}^{n} \big[a_{-i}(x_i) = y_i\big],$$

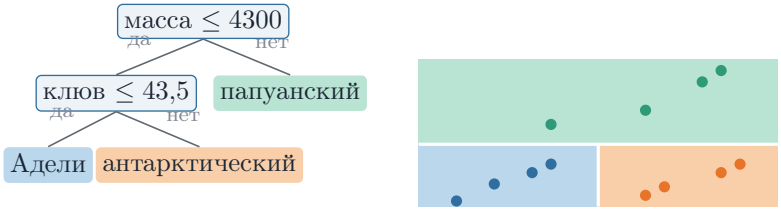
где a_{-i} — модель, построенная без объекта i. Из нескольких k берут то, у которого оценка скользящего контроля лучше.

> **Ловушка.** Отложенный объект не должен быть соседом самому себе. Если искать соседей x_i среди всех объектов, ближайшим окажется он сам, на расстоянии 0, и при $k = 1$ модель «угадает» каждый объект: LOO выйдет 1 при любых данных, где нет одинаковых объектов с

разными ответами. Это та же проверка на train, только переодетая.

3 Решающие деревья

Решающее дерево задаёт вопросы по одному признаку: «$x_j \le t$?». От ответа зависит, какой вопрос следующий, а в конце пути — *лист* с ответом. Слева — дерево для пингвинов, которое построим ниже, справа — как оно делит плоскость признаков:



Вопрос делит объекты узла на две части, и хороший вопрос делит так, что в каждой части ответы почти одинаковы. Неоднородность узла меряют *примесью*. Пусть доли классов в узле — $p_0, \dots, p_{K-1}$; *критерий Джини* и *энтропия* —

$$G = 1 - \sum_c p_c^2, \qquad H = -\sum_c p_c \log_2 p_c,$$

где слагаемое с $p_c = 0$ считают нулём. Обе равны 0 в чистом узле, где один класс, и наибольшие, когда классы поровну: при K равных классах $G = 1 - 1/K$ и $H = \log_2 K$. У Джини есть смысл: $\sum_c p_c^2$ — вероятность, что два объекта, взятые из

узла наугад с возвращением, одного класса, а G — что разных.

```
def gini(y):
    p = np.bincount(y) / len(y)
    return 1 - (p ** 2).sum()

def entropy(y):
    p = np.bincount(y) / len(y)
    p = p[p > 0]
    return -(p * np.log2(p)).sum()

print(gini(y), entropy(y))
print(gini(np.array([1, 1, 1, 2])))
```

```
0.6666666666666667 1.584962500721156
0.375
```

Прирост — насколько вопрос уменьшил примесь: из примеси узла вычитают примеси частей, взвешенные их долями. Если в узле n объектов, а вопрос отправил n_L из них влево и n_R вправо,

$$\Delta = I(\text{узел}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R),$$

где I — Джини или энтропия. Лучший вопрос — с наибольшим приростом.

Какие пороги перебирать? Между двумя соседними значениями признака любой порог делит объекты одинаково, поэтому достаточно одного порога на промежуток. Берут середину: порог одинаково далёк от обеих сторон. У признака с m различными значениями выходит $m - 1$ порогов. Перебор для массы на всех двенадцати пингвинах:

```
def gain(x, y, t, crit=gini):
    L, R = y[x <= t], y[x > t]
    n = len(y)
    return (crit(y) - len(L) / n * crit(L)
```

```
            - len(R) / n * crit(R))

v = np.unique(X[:, 1])
for t in (v[1:] + v[:-1]) / 2:
    print(t, round(gain(X[:, 1], y, t), 4))
```

```
3350.0 0.0606
3475.0 0.0333
3575.0 0.0741
3700.0 0.0833
3875.0 0.1667
4300.0 0.3333
4775.0 0.2222
5150.0 0.1333
5500.0 0.0606
```

Лучший порог по массе — 4300, середина между 3950 и 4650: справа четыре папуанских, слева все остальные, прирост $\frac{2}{3} - \frac{8}{12} \cdot \frac{1}{2} = \frac{1}{3}$. Тот же перебор для клюва:

```
v = np.unique(X[:, 0])
ts = (v[1:] + v[:-1]) / 2
g = [gain(X[:, 0], y, t) for t in ts]
print(ts[np.argmax(g)], round(max(g), 4))
```

```
43.5 0.2476
```

Пример (дерево для пингвинов). Клюв в корне проигрывает массе, и корень — вопрос «масса ≤ 4300?». Правая часть — четыре папуанских, чистый лист. В левой по четыре Адели и антарктических, $G = \frac{1}{2}$, и перебор в ней начинается заново, только по её восьми объектам. Масса здесь бесполезна, у двух видов она одинаковая, а вопрос «клюв $\leq 43{,}5$?» делит узел начисто, с приростом $\frac{1}{2}$. Оба листа чистые, и дерево готово: глубина 2, три листа. На тесте оно не ошибается: пингвин $(41, 3520)$ лёгкий, с коротким клювом — Адели, $(45, 3450)$ лёгкий, с длинным — антарктический, $(44, 4750)$ тяжёлый — папуанский.

Так строится любое дерево — *жадно*: в корне выбрать лучший вопрос по всем признакам и порогам, разделить объекты и в каждой части повторить то же, как будто она — вся выборка. Жадно — потому что вопрос выбирают по приросту сейчас, не заглядывая, что он даст ниже. Узел становится листом, когда в нём один класс, когда никакой вопрос его не делит или когда сработал *останов*: достигнута наибольшая глубина, в узле меньше заданного числа объектов, прирост меньше заданного. Ответ листа — самый частый класс его объектов, а доли классов в листе — оценка их вероятностей.

Масштаб дереву не важен: вопрос «$x_j \le t$?» после стандартизации — тот же вопрос с другим t. Это отличает деревья от соседей и, как дальше выяснится, от линейных моделей.

Дерево регрессии

В регрессии примесь узла — дисперсия ответа в нём, то есть MSE ответа «среднее узла», а ответ листа — среднее его y. Функция `gain` подходит и сюда: вместо `gini` — `np.var`. Масса по длине крыла, лучший первый вопрос и ответы двух листов:

```
v = np.unique(fl)
ts = (v[1:] + v[:-1]) / 2
g = [gain(fl, m, t, crit=np.var) for t in ts]
best = ts[np.argmax(g)]
left, right = m[fl <= best], m[fl > best]
print(best, left.mean(), right.mean())
```

```
205.5 3668.75 5137.5
```

Глубина и переобучение

Без останова дерево растёт, пока листья не станут чистыми, и на train не ошибается вовсе, если в нём нет одинаковых

объектов с разными ответами. Глубокие листья держат по одному-два объекта — это уже не закономерность, а запомненный шум. Мелкое дерево, наоборот, слишком грубое. Глубина — главный гиперпараметр дерева, и выбирают её скользящим контролем; опыт с глубиной — в разделе 6.

4 Линейные модели

Линейная регрессия: $a(x) = w_1 x_1 + \cdots + w_d x_d + b = w \cdot x + b$. Веса w и *свободный член* b подбирают так, чтобы MSE на train была наименьшей, — это *метод наименьших квадратов*.

Свободный член прячут в веса: к каждому объекту приписывают признак, всегда равный 1, и b становится его весом. Дальше в этой записи: X — таблица $n \times (d+1)$ со столбцом единиц, $w \in \mathbb{R}^{d+1}$, ответы модели на всех объектах — вектор Xw, и

$$L(w) = \frac{1}{n}\sum_{i=1}^{n}(x_i \cdot w - y_i)^2 = \frac{1}{n}\|Xw - y\|^2.$$

Утверждение. Минимум L достигается на решениях *нормальных уравнений* $X^\top X w = X^\top y$. Если столбцы X линейно независимы, решение одно: $w = (X^\top X)^{-1} X^\top y$.

Почему так. Производная по весу w_j —

$$\frac{\partial L}{\partial w_j} = \frac{2}{n}\sum_{i=1}^{n}(x_i \cdot w - y_i)\, x_{ij},$$

и все производные вместе — *градиент* $\nabla L = \frac{2}{n} X^\top (Xw - y)$: координата j вектора $X^\top r$ — сумма произведений столбца j на r. В минимуме градиент нулевой, отсюда уравнения. Обратно, пусть $X^\top X w^* = X^\top y$. Для любого v

$$\|X(w^* + v) - y\|^2 = \|Xw^* - y\|^2 + 2v^\top X^\top (Xw^* - y) + \|Xv\|^2,$$

среднее слагаемое равно нулю, последнее не меньше нуля, и $L(w^* + v) \geq L(w^*)$ — с равенством только при $Xv = 0$. При

независимых столбцах $Xv = 0$ лишь для $v = 0$: минимум один. И $X^\top X$ тогда обратима: из $X^\top Xv = 0$ следовало бы $\|Xv\|^2 = v^\top X^\top Xv = 0$.

Масса по длине крыла: столбец единиц, столбец крыла и система `solve`. Обратную матрицу не считают: `solve` решает систему точнее и быстрее.

```
A = np.column_stack([np.ones(12), fl])
w = np.linalg.solve(A.T @ A, A.T @ m)
print(w.round(2))
print(((A @ w - m) ** 2).mean().round(1))
```

```
[-6725.05    54.37]
29890.7
```

Каждый миллиметр крыла — 54 грамма массы. При одном признаке уравнения решаются и руками. Первое, по столбцу единиц, — $\sum_i (wx_i + b - y_i) = 0$, то есть $b = \bar{y} - w\bar{x}$: прямая проходит через точку средних. Подставим во второе, $\sum_i (wx_i + b - y_i)\, x_i = 0$:

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

Гребневая регрессия

Если признаки почти зависимы — скажем, крыло в миллиметрах и оно же в сантиметрах с округлением, — $X^\top X$ почти вырождена. Веса тогда огромны, противоположных знаков и сильно меняются от малой правки данных. *Гребневая регрессия*, ridge, добавляет к ошибке штраф за большие веса:

$$L_\lambda(w) = \|Xw - y\|^2 + \lambda\|w\|^2, \qquad (X^\top X + \lambda E)\, w = X^\top y.$$

Уравнение выводится так же: градиент штрафа — $2\lambda w$. При $\lambda > 0$ матрица обратима всегда: $v^\top (X^\top X + \lambda E) v = \|Xv\|^2 + \lambda\|v\|^2 > 0$ при $v \neq 0$. Ошибка

здесь — сумма квадратов, а не среднее: так её пишет sklearn, а переход к среднему — деление λ на n.

В формуле штрафуются все веса. На деле свободный член не штрафуют. Штраф тянет веса к нулю, а стянутый к нулю b сдвигает к нулю все ответы — и результат зависит от того, откуда отсчитывать y: от нуля граммов или от трёх килограммов. От сдвига y не зависят только наклоны, и штраф на них осмыслен. Сами наклоны зависят от единиц признаков, поэтому перед ridge признаки стандартизуют.

Градиентный спуск

Нормальные уравнения — система $(d+1) \times (d+1)$, и при тысячах признаков её решать дорого, а у большинства моделей готовой формулы нет вовсе. Общий способ — *градиентный спуск*, шаг против градиента:

$$w \leftarrow w - \eta \nabla L(w),$$

где $\eta > 0$ — *шаг обучения*. Градиент показывает, куда L растёт быстрее всего, и малый шаг против него уменьшает L. Спуск для массы по крылу, признак стандартизован:

```
s = (fl - fl.mean()) / fl.std()
A = np.column_stack([np.ones(12), s])
w = np.zeros(2)
for step in range(100):
    w = w - 0.1 * 2 / 12 * A.T @ (A @ w - m)
print(w.round(2))
print(np.linalg.solve(A.T @ A, A.T @ m).round(2))
```

```
[4158.33  729.75]
[4158.33  729.75]
```

Спуск пришёл туда же, куда `solve`. Стандартизация здесь не для красоты. У стандартного признака среднее 0 и средний квадрат 1, поэтому $\frac{1}{n}A^\top A = E$, градиент равен $2(w - w^*)$, и шаг умножает расстояние до ответа w^* на $1 - 2\eta = 0{,}8$: за сто

шагов — в $0{,}8^{100} \approx 2 \cdot 10^{-10}$ раз. При $\eta > 1$ множитель по модулю больше 1, и спуск разлетается. В миллиметрах у $\frac{1}{n} A^\top A$ собственные числа около $4 \cdot 10^4$ и около $4 \cdot 10^{-3}$; шаг, при котором спуск не разлетается вдоль первого направления, вдоль второго двигает на миллионные доли за шаг.

Категория: номер или индикаторы

Признак-категория — остров, где живёт пингвин: Бискоу, Дрим или Торгерсен. Записать его числами 0, 1, 2 — значит сказать линейной модели, что острова стоят на прямой в этом порядке и масса от острова к острову меняется на одно и то же. Так у модели и выйдет, хотя порядок номеров случаен. Правильно — *индикаторы*, one-hot: по столбцу на значение, 1 в строках этого острова и 0 в остальных. Тогда у каждого острова свой вес.

```
isl = np.array([2, 1, 0, 2, 1, 1, 1, 1, 0, 0, 0, 0])
A = np.column_stack([np.ones(12), isl])
w = np.linalg.solve(A.T @ A, A.T @ m)
print((w[0] + w[1] * np.arange(3)).round(1))
H = np.eye(3)[isl]
print(np.linalg.solve(H.T @ H, H.T @ m))
```

```
[4617.3 4005.3 3393.3]
[4770. 3700. 3775.]
```

`np.eye(3)[isl]` — строки единичной матрицы по номерам, то есть готовые индикаторы. С ними ответ на каждом острове — средняя масса его пингвинов, а с номерами ответы стоят на одной прямой, и Торгерсену досталось 3393 вместо 3775. Столбца единиц рядом с индикаторами нет: в каждой строке их сумма и так 1, и вместе со свободным членом столбцы зависимы, $H^\top H$ вырождена. Поэтому либо свободного члена нет, либо один индикатор выбрасывают.

Логистическая регрессия

Для классов 0 и 1 линейная модель считает *отступ* $z = w \cdot x$ (свободный член — внутри), а *сигмоида* переводит его в вероятность класса 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad p = \sigma(w \cdot x).$$

σ растёт от 0 до 1, $\sigma(0) = \frac{1}{2}$ и $\sigma(-z) = 1 - \sigma(z)$. Класс 1 предсказывают при $p > \frac{1}{2}$, то есть при $w \cdot x > 0$: граница классов — гиперплоскость $w \cdot x = 0$. Это *логистическая регрессия*: классификатор, хотя и зовётся регрессией.

Веса подбирают по *логистической функции потерь*, log-loss:

$$L(w) = -\frac{1}{n} \sum_{i=1}^{n} \big(y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \big), \qquad p_i = \sigma(w \cdot x_i).$$

У каждого объекта работает одно слагаемое: $-\ln p_i$ у класса 1 и $-\ln(1 - p_i)$ у класса 0 — минус логарифм вероятности, которую модель дала верному ответу. Уверенная верная модель платит почти 0, уверенная неверная — очень много. Если объекты независимы, nL — минус логарифм вероятности всех ответов train, и минимум log-loss — веса, при которых наблюдённые ответы вероятнее всего.

Утверждение. $\nabla L(w) = \frac{1}{n} X^\top (p - y)$, где p — вектор вероятностей p_i.

Почему так. Сначала производная сигмоиды:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z) \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big).$$

Слагаемое одного объекта — $\ell = -y \ln p - (1 - y) \ln(1 - p)$ при $p = \sigma(z)$, и

$$\frac{d\ell}{dz} = -\frac{y}{p}\, p(1-p) + \frac{1 - y}{1 - p}\, p(1-p) = -y(1-p) + (1-y)p = p - y.$$

А $\partial z/\partial w_j = x_j$, поэтому $\partial \ell/\partial w_j = (p - y)\, x_j$, и сумма по объектам — $X^\top (p - y)$. Та же форма, что у MSE: «ошибка на признак», только ошибка — разность вероятности и ответа.

Уравнение $X^\top (p - y) = 0$ формулой не решается, и веса ищут спуском, $w \leftarrow w - \eta \cdot \frac{1}{n} X^\top (p - y)$. L выпукла, и при подходящем шаге спуск приходит к минимуму — если минимум есть. Пример: «папуанский или нет» по стандартизованной массе, свободный член 0, вес от 1 до 8:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

t = (y == 2).astype(int)
A = np.column_stack([np.ones(12), Z[:, 1]])
for c in [1, 2, 4, 8]:
    p = sigmoid(A @ np.array([0, c]))
    loss = -(t * np.log(p) + (1 - t) * np.log(1 - p))
    print(c, loss.mean().round(4))
```

```
1 0.3734
2 0.2156
4 0.0876
8 0.0216
```

Ловушка. Чем больше вес, тем меньше потери, и так без конца: здесь классы разделимы — папуанские тяжелее всех, — и log-loss можно сделать сколь угодно малым, растягивая веса. Минимума нет, и спуск растит веса, пока его не остановят. Спасает штраф, как в ridge, и sklearn ставит его по умолчанию — раздел 6.

И ещё одна беда больших отступов — `np.exp` переполняется:

```
print(sigmoid(np.array([-800.0, 0.0, 800.0])))
```

```
[0.  0.5 1. ]
RuntimeWarning: overflow encountered in exp
```

e^{800} в дробные числа не помещается, выходит бесконечность и предупреждение. σ при этом верна, 0, но $\ln 0 = -\infty$, и log-loss с такой вероятностью тоже бесконечен. Поэтому в библиотеках log-loss считают формулами, которые обходят переполнение.

Перцептрон

Перцептрон — самый старый линейный классификатор. Классы здесь -1 и $+1$, ответ — знак $w \cdot x + b$. Обучение — проход по объектам по порядку: если объект (x_i, y_i) классифицирован неверно или лежит на границе, то есть $y_i(w \cdot x_i + b) \le 0$, веса поправляют:

$$w \leftarrow w + y_i x_i, \qquad b \leftarrow b + y_i.$$

Проходы повторяют, пока в проходе не будет ни одной поправки. Поправка сдвигает отступ ошибочного объекта в нужную сторону: новое $y_i(w \cdot x_i + b)$ больше старого на $\|x_i\|^2 + 1$.

Пример (перцептрон руками). Объекты $x_1 = (2, 0)$ класса $+1$, $x_2 = (0, 3)$ класса -1, $x_3 = (2, 2)$ класса $+1$; старт — $w = (0, 0)$, $b = 0$.

- x_1: отступ 0 — поправка, $w = (2, 0)$, $b = 1$.
- x_2: $w \cdot x_2 + b = 1$, а класс -1 — поправка, $w = (2, -3)$, $b = 0$.
- x_3: $4 - 6 + 0 = -2$, а класс $+1$ — поправка, $w = (4, -1)$, $b = 1$.
- Второй проход: $w \cdot x_i + b$ равны 9, -2, 7 при классах $+1$, -1, $+1$. Знаки верные, поправок нет.

Граница — прямая $4x_1 - x_2 + 1 = 0$. Если классы разделимы гиперплоскостью, перцептрон останавливается за конечное число поправок (теорема Новикова); если нет, поправки не кончаются никогда, и число проходов ограничивают.

5 k-средних

Бывает, что ответов нет и в объектах надо найти группы — *кластеры*. Метод *k-средних* ищет k центров $\mu_1, \ldots, \mu_k$ и номер кластера c_i у каждого объекта так, чтобы сумма квадратов расстояний от объектов до центров их кластеров была мала:

$$\text{SSE} = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2.$$

Разбиений около k^n, их не перебрать. *Алгоритм Ллойда* улучшает по очереди то разбиение, то центры:

1. *назначение*: каждый объект — в кластер ближайшего центра;
2. *пересчёт*: каждый центр — в среднее объектов своего кластера.

Шаги повторяют, пока назначение не перестанет меняться.

Назначение — таблица квадратов расстояний «объект — центр» и номер наименьшего в каждой строке. Пингвины в стандартных единицах, стартовые центры — пингвины 0, 4 и 8:

```
C = Z[[0, 4, 8]]
diff = Z[:, None, :] - C[None, :, :]
D = (diff ** 2).sum(axis=2)
c = D.argmin(axis=1)
print(D.shape, c)
print(D.min(axis=1).sum().round(3))
```

```
(12, 3) [0 0 0 0 1 1 1 1 2 2 2 2]
6.121
```

Последняя строка — SSE этого назначения: у каждого объекта — квадрат расстояния до своего центра. Пересчёт — среднее объектов по каждому кластеру. Если в кластер не попал ни один объект, кластер *пуст*, и среднее пустого множества не определено: что делать с его центром —

оставить на месте, убрать кластер или перенести в далёкий объект, — отдельное правило алгоритма.

Пример (на прямой). Объекты 1, 2, 4, 7, 8, 12, $k = 2$, стартовые центры — 1 и 2. В строке — кластеры после назначения и SSE при старых центрах, затем центры после пересчёта и SSE при них:

шаг	назначение	SSE	центры	SSE
1	$\{1\}$, $\{2, 4, 7, 8, 12\}$	165	1; 6,6	59,2
2	$\{1, 2\}$, $\{4, 7, 8, 12\}$	39,04	1,5; 7,75	33,25
3	$\{1, 2, 4\}$, $\{7, 8, 12\}$	25,4375	$\frac{7}{3}$; 9	$18\frac{2}{3}$

Четвёртое назначение ничего не меняет: 7 ближе к 9, чем к $\frac{7}{3}$. Останов.

Утверждение. SSE не растёт ни при назначении, ни при пересчёте. Если при равных расстояниях объект остаётся в своём кластере, алгоритм останавливается за конечное число шагов.

Почему так. Назначение: центры неподвижны, и каждый объект выбирает ближайший, так что его слагаемое не больше прежнего. Пересчёт: разбиение неподвижно, и слагаемые кластера S — это $\sum_{i \in S} \|x_i - \mu\|^2$. Среднее $\bar{x}$ даёт наименьшую из таких сумм: для любого μ

$$\sum_{i \in S} \|x_i - \mu\|^2 = \sum_{i \in S} \|x_i - \bar{x}\|^2 + |S| \cdot \|\bar{x} - \mu\|^2,$$

потому что $\|x_i - \mu\|^2 = \|x_i - \bar{x}\|^2 + 2(x_i - \bar{x}) \cdot (\bar{x} - \mu) + \|\bar{x} - \mu\|^2$, а $\sum_{i \in S}(x_i - \bar{x}) = 0$. Останов: если назначение изменилось, какой-то объект перешёл к строго более близкому центру, и SSE строго уменьшилась. Значит, разбиения не повторяются, а их конечное число.

Локальные минимумы и старты

SSE не растёт, но приходит алгоритм в *локальный минимум* — состояние, которое шаги не меняют, а SSE не наименьшая. Объекты 0, 1, 10, 11, 20, 21 и $k = 3$. Из старта в объектах 0, 1 и 11 центры приходят в 0, 1 и 15,5 и стоят: объект 10 ближе к 15,5, чем к 1. SSE этого состояния — 101, а у разбиения на пары — 1,5.



Поэтому k-средних запускают много раз с разных стартов и берут результат с наименьшей SSE. Старт выбирают случайно: k случайных объектов или *k-means++* — первый центр случайный объект, каждый следующий — объект, взятый с вероятностью, пропорциональной квадрату расстояния до ближайшего уже выбранного центра. Так центры разносятся по данным, и старты вроде показанного редки.

k-средних считает те же расстояния, что и соседи, и масштаб ему важен так же: в граммах пингвины разбились бы по одной массе. Признаки стандартизуют.

Как сравнить два разбиения

Найденные кластеры хочется сравнить с видами, но номера кластеров произвольны: разбиения $(0, 0, 1, 1)$ и $(1, 1, 0, 0)$ — одно и то же. Поэтому сравнивают пары объектов. Разбиения согласны на паре, если в обоих она вместе или в обоих врозь; доля согласных пар — *индекс Рэнда*. Недостаток у него такой: случайные разбиения тоже согласны на многих парах — больше всего на парах «врозь», и индекс у них далёк от нуля.

Скорректированный индекс Рэнда, ARI, вычитает то, что дала бы случайность. Пусть n_{ij} — сколько объектов в кластере i первого разбиения и в кластере j второго, a_i и b_j — суммы этой таблицы по строкам и столбцам, а $\binom{m}{2} = m(m-1)/2$ — число пар из m. Тогда

$$\mathrm{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - E}{\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - E}, \qquad E = \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}.$$

В числителе — сколько пар вместе в обоих разбиениях, минус сколько их вышло бы в среднем, если перемешать объекты при тех же размерах кластеров; в знаменателе — то же для верхней границы этого числа. ARI $= 1$, когда разбиения совпадают с точностью до номеров, около 0 — когда они не связаны, и бывает отрицательным.

Пример (шесть объектов). Разбиения $(0,0,0,1,1,1)$ и $(0,0,1,1,2,2)$. Таблица n_{ij} — строки $(2,1,0)$ и $(0,1,2)$; вместе в обоих — $1+1=2$ пары. $a=(3,3)$, $b=(2,2,2)$: $\sum\binom{a_i}{2}=6$, $\sum\binom{b_j}{2}=3$, $E = 6\cdot 3/15 = \frac{6}{5}$. ARI $= (2-\frac{6}{5})/(\frac{9}{2}-\frac{6}{5}) = \frac{8}{33} \approx 0{,}24$, а индекс Рэнда — $\frac{10}{15}=\frac{2}{3}$: из 15 пар две вместе в обоих и восемь врозь в обоих.

6 То же в sklearn

У каждой модели sklearn один порядок: создать объект с гиперпараметрами, `fit(X, y)` — обучить, `predict(X)` — предсказать. Выученные числа лежат в полях с подчёркиванием на конце: `coef_`, `cluster_centers_`.

Преобразования признаков устроены так же, только вместо `predict` — `transform`.

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

sc = StandardScaler().fit(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(sc.transform(X), y)
print(sc.mean_.round(1), sc.scale_.round(1))
print(model.predict(sc.transform(Xt)))
```

```
[  44.6 4158.3] [  4.9 750. ]
[0 0 1 1 2 2]
```

Правило. `fit` у преобразования — только на train, а к test — `transform` с теми же числами. `fit_transform` на test — та самая ошибка, от которой предостерегает раздел 2.

sklearn ждёт таблицу и для одного признака: столбец `x[:, None]`, а не ряд `x`. Расстояние L1 — параметр `p=1`, взвешенные соседи — `weights="distance"`, регрессия — `KNeighborsRegressor`.

Ловушка. Ничьи у `KNeighborsClassifier`. При равных голосах он берёт меньший номер класса, как `argmax`. При равных расстояниях порядок соседей не обещан: кто из равноудалённых войдёт в k, зависит от способа поиска, `algorithm`. Сетка 7×7 целых точек, классы 0 и 1 через

одну, вопрос — в центре клетки, на равном расстоянии от четырёх её углов:

```
g = np.arange(-3, 4)
P = np.array([(i, j) for i in g for j in g])
lab = np.arange(49) % 2
for alg in ["brute", "kd_tree"]:
    nn = KNeighborsClassifier(1, algorithm=alg)
    nn.fit(P, lab)
    print(alg, nn.predict([[0.5, 0.5]]))
```

```
brute [1]
kd_tree [0]
```

Один и тот же сосед, две реализации поиска — два разных ответа. Где правило ничьей важно — а в задаче оно задано условием, — соседей считают сами.

Деревья

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text

tree = DecisionTreeClassifier(max_depth=2)
tree.fit(X, y)
names = ["клюв", "масса"]
print(export_text(tree, feature_names=names))
```

```
|--- масса <= 4300.00
|   |--- клюв <= 43.50
|   |   |--- class: 0
|   |--- клюв >  43.50
|   |   |--- class: 1
|--- масса >  4300.00
|   |--- class: 2
```

Это дерево из раздела 3. Критерий — `criterion="gini"` по умолчанию или `"entropy"`; останов — `max_depth`,

`min_samples_leaf`, `min_impurity_decrease`. Пороги — середины, как у нас, но признаки sklearn хранит в `float32`, и середина берётся от чисел, уже округлённых до семи значащих цифр.

Опыт с глубиной. Двести случайных точек, класс — знак $x_1 + x_2$, но у каждой пятой точки ответ перевёрнут: это шум. Сто точек — train, сто — test; `score` — доля верных:

```
rng = np.random.default_rng(0)
P = rng.normal(size=(200, 2))
c = (P.sum(axis=1) > 0).astype(int)
c[::5] = 1 - c[::5]
for depth in [1, 2, 4, 8, None]:
    tr = DecisionTreeClassifier(max_depth=depth,
                                random_state=0)
    tr.fit(P[:100], c[:100])
    print(depth, tr.score(P[:100], c[:100]),
          tr.score(P[100:], c[100:]))
```

```
1 0.71 0.61
2 0.82 0.67
4 0.83 0.65
8 0.93 0.63
None 1.0 0.58
```

Чем глубже, тем лучше на train, до безошибочного дерева без останова; на test — сначала лучше, потом хуже: дерево учит шум.

Ловушка. Ничьи у дерева. Если два вопроса дают одинаковый прирост, sklearn берёт тот, что встретил первым, а признаки он перебирает в случайном порядке — даже когда смотрит их все. Порядок задаёт

`random_state`, и без него дерево на тех же данных бывает разным. Два одинаковых столбца массы:

```
D2 = np.column_stack([X[:, 1], X[:, 1]])
for seed in range(6):
    tr = DecisionTreeClassifier(max_depth=1,
                                random_state=seed)
    print(tr.fit(D2, y).tree_.feature[0], end=" ")
```

```
1 1 0 0 0 1
```

Номер признака в корне меняется от зерна.

Дерево регрессии — `DecisionTreeRegressor`, примесь — MSE, как у нас:

```
from sklearn.tree import DecisionTreeRegressor

reg = DecisionTreeRegressor(max_depth=1)
reg.fit(fl[:, None], m)
print(reg.tree_.threshold[0])
print(reg.predict([[190], [215]]))
```

```
205.5
[3668.75 5137.5 ]
```

Линейные модели

`LinearRegression` решает тот же метод наименьших квадратов, свободный член — `intercept_`, веса — `coef_`.

`Ridge(alpha=...)` штрафует $\alpha\|w\|^2$ при сумме квадратов ошибок и свободный член не трогает:

```
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge

lr = LinearRegression().fit(fl[:, None], m)
print(lr.coef_.round(2), lr.intercept_.round(2))
for a in [1, 100, 10000]:
    r = Ridge(alpha=a).fit(s[:, None], m)
    print(a, r.coef_.round(1), r.intercept_.round(1))
```

```
[54.37] -6725.05
1 [673.6] 4158.3
100 [78.2] 4158.3
10000 [0.9] 4158.3
```

С ростом штрафа наклон уходит к нулю, а свободный член остаётся средней массой: модель сжимается к константе, а не к нулю.

`LogisticRegression` тоже штрафует по умолчанию: минимизирует $\frac{1}{2}\|w\|^2 + C\sum_i \ell_i$ с $C = 1$ — сумма потерь, не среднее, и свободный член без штрафа. Поэтому веса у неё

меньше, чем у чистого log-loss, а на разделимых данных конечны. Без штрафа — `C=np.inf`.

```
from sklearn.linear_model import LogisticRegression

zm = Z[:, [1]]
for C in [1, 100, np.inf]:
    lg = LogisticRegression(C=C).fit(zm, t)
    w, b = lg.coef_[0, 0], lg.intercept_[0]
    print(C, round(w, 2), round(b, 2))
lg = LogisticRegression().fit(zm, t)
print(lg.predict_proba(Zt[3:, [1]]).round(3))
```

```
1 1.62 -0.92
100 6.84 -1.74
inf 16.05 -3.31
[[0.921 0.079]
 [0.227 0.773]
 [0.411 0.589]]
```

Без штрафа на разделимых данных вес большой и конечный лишь потому, что спуск внутри остановился по своему допуску. `predict_proba` — вероятности классов 0 и 1 по столбцам, здесь у трёх последних пингвинов теста. Для K классов у модели по набору весов на класс, и вместо сигмоиды — softmax.

k-средних

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

for data in [X, Z]:
    km = KMeans(n_clusters=3, n_init=10,
                random_state=0).fit(data)
    ari = adjusted_rand_score(y, km.labels_)
    print(km.labels_, round(ari, 3))
print(round(km.inertia_, 3))
```

```
[0 0 0 0 0 0 0 0 2 1 2 1] 0.368
[0 0 0 0 1 1 1 1 2 2 0 2] 0.737
4.417
```

В граммах кластеры — полосы по массе, и с видами они совпадают плохо. В стандартных единицах — почти виды: папуанский с коротким клювом, (41, 4650), ушёл к Адели. SSE этого разбиения, она лежит в `inertia_`, меньше, чем у разбиения по видам, 5,079: k-средних ищет компактные группы, а не виды, и ARI эту разницу видит. Центры — в `cluster_centers_`. Свой старт — массив центров в `init`:

```
line = np.array([[0], [1], [10], [11], [20], [21]])
start = np.array([[0], [1], [11]])
for init in [start, "k-means++"]:
    km = KMeans(3, init=init, n_init=1,
                random_state=0).fit(line)
    print(km.cluster_centers_.ravel(), km.inertia_)
```

```
[ 0.   1.  15.5] 101.0
[10.5  0.5 20.5] 1.5
```

Ловушка. Правила `KMeans` отличаются от точного алгоритма Ллойда.

- Старты: `n_init` по умолчанию `"auto"` — для k-means++ это *один* запуск; старт случайный, его фиксирует `random_state`.
- Останов: когда назначение не изменилось *или* когда центры за шаг сдвинулись мало — сумма квадратов сдвигов не больше `tol` (10^{-4}), умноженного на среднюю дисперсию признаков; и не больше `max_iter=300` шагов. Остановившись по `tol`, он ещё раз назначает объекты, но центры не пересчитывает. Точное «до неизменного назначения» — `tol=0`.
- Пустой кластер sklearn не оставляет: переносит его центр в объект, самый далёкий от своего центра.

7 Итог

- *Соседи* ничего не учат, хранят таблицу; гиперпараметры — k, расстояние, веса. Масштаб важен.
- *Дерево* учит вопросы «$x_j \le t$?» жадно, по приросту Джини или энтропии; гиперпараметры — глубина и останов. Масштаб не важен.
- *Линейные модели* учат веса: МНК — нормальными уравнениями или спуском, логистическая — спуском по log-loss, перцептрон — поправками на ошибках; гиперпараметры — штраф и шаг. Масштаб важен для штрафа и для спуска.
- *k-средних* учит центры шагами Ллойда, SSE не растёт, но минимум бывает локальным; гиперпараметры — k и старты. Масштаб важен.

У всех моделей качество меряют на отложенных данных, гиперпараметры выбирают скользящим контролем, а правило ничьих — часть модели. Дальше по курсу — метрики

качества по файлу предсказаний и ансамбли: лес и бустинг строятся из деревьев этого урока.